

Question 1: Shape Hierarchy with Area Comparison (Friend Function)

Create a base class `Shape` with a protected data member `name` (string). Derive two classes, `Circle` (private member `radius`) and `Rectangle` (private members `length`, `width`).

- Both derived classes should have a public method `double area()` that returns the area.
 - Implement a **friend function** `bool compareArea(const Shape& s1, const Shape& s2)` that compares the areas of any two shapes and returns `true` if `s1` has a larger area than `s2`.
 - The friend function must directly access the private dimension members of `Circle` and `Rectangle` (use `dynamic_cast` and check types) – do **not** rely on the `area()` method inside the friend.
 - Demonstrate in `main()` with two circles and two rectangles, and print the result of comparisons.
-

Question 2: Abstract Employee Payroll with Friend Display

Design an **abstract base class** `Employee` with:

- Pure virtual function `double calculateSalary() const = 0`.
- Protected data members `id` (int) and `name` (string).

Derive two concrete classes:

- `HourlyEmployee`: private `hourlyRate`, `hoursWorked`; implements `calculateSalary` as `rate × hours`.
- `SalariedEmployee`: private `monthlySalary`; implements `calculateSalary` returning the monthly salary.

Add a **friend function** `void displayEmployeeDetails(const Employee& emp)` that prints the employee's ID, name, and calls `calculateSalary()` to print the salary. Because the friend needs the salary but should not access private details directly, it may use the public interface – but *also* declare it as a friend so that in the future it could access private helpers if needed.

In `main()`, create a vector of `Employee*` containing both hourly and salaried employees, and call `displayEmployeeDetails` for each.

Question 3: Bank Account Inheritance with Friend Transfer Function

Create a base class `Account` with:

- Private `accountNumber` (int) and `balance` (double).
- Public constructor, getter for account number, and a virtual destructor.
- Declare `void transfer(Account& from, Account& to, double amount)` as a **friend function**.

Derive `SavingsAccount` from `Account` with:

- Private `interestRate` (double).
- Public method `void applyInterest()` that increases balance by $(\text{balance} \times \text{interestRate})$.

The friend function `transfer` must:

- Check that `from.balance >= amount`, then subtract from `from.balance` and add to `to.balance` (accessing private members directly).
- Print success/failure message.

In `main()`, create two `SavingsAccount` objects, apply interest to one, transfer funds between them, and display final balances using public getters (or a display method).

Question 4: Multiple Inheritance: WorkingStudent with Friend Printer

Define a class `Person` with private members `name` (string) and `age` (int), and a public parameterized constructor.

Derive two classes from `Person`:

- `Student` (adds private `studentID` and `major`)
- `Employee` (adds private `employeeID` and `department`)

Now create a class `WorkingStudent` that **multiply inherits** from both `Student` and `Employee`. (Use virtual inheritance from `Person` to avoid ambiguity.)

- `WorkingStudent` constructor should take all information and initialize base classes appropriately.

Implement a **friend function** `void printWorkingStudentInfo(const WorkingStudent& ws)` that prints all details: name, age, studentID, major, employeeID, and department. The function must access the private members of `Person`, `Student`, and `Employee` directly. Test in `main()` by creating a `WorkingStudent` object and calling the friend printer.

Question 5: Abstract Series with Friend Summation Helper

Design an **abstract class** `Series` with:

- Pure virtual function `double term(int n) const = 0` that returns the n th term.
- Protected integer `limit` (number of terms).
- A public member function `double sum()` that computes the sum of terms from 1 to `limit` by calling `term()` in a loop.

Derive two concrete classes:

- `ArithmeticSeries`: private `firstTerm` and `commonDifference`; implements `term(n)` as `firstTerm + (n-1)*commonDifference`.
- `GeometricSeries`: private `firstTerm` and `commonRatio`; implements `term(n)` as `firstTerm * pow(commonRatio, n-1)`.

Create a **friend function** `bool isSumGreater(const Series& a, const Series& b)` that compares the total sums of two series. This friend will call the public `sum()` method – but it must also be allowed to directly inspect the `limit` member (protected) and, if needed, the private data of derived classes (e.g., using `dynamic_cast`). In the implementation, simply use `sum()` and print whether the first series' sum is greater.

Write a `main()` that creates an `ArithmeticSeries` and a `GeometricSeries`, computes their sums, and uses the friend to compare them.